

总线与 I/O 硬件

同步处理器怎样与异步设备完成可确认的协作

408 · 计算机组成原理 Topic CO-08

母问题：一次设备工作怎样从意图变成可确认的完成？

总线、控制器、轮询、中断和 DMA 不是五个孤立名词，而是一条责任链：

Request → Interface State → Arbitration → Transaction → Transfer → Completion Signal

每一步都要回答：谁拥有控制权，谁驱动共享资源，数据经过哪里，什么状态证明本步结束。

本册学习范围

- 本册解释多个主设备如何共享总线，设备控制器怎样接受请求、搬运数据并报告完成，以及查询、中断和 DMA 的硬件差异。
- 本册重点解释事务四阶段、同步/异步握手、集中式/分布式仲裁、I/O 端口、中断入口、DMA 传送模式、通道和有效带宽。
- 驱动程序、请求队列、阻塞/唤醒、缓冲区管理和设备调度属于软件；本册只说明硬件交给软件的状态和完成证据。

1 术语先行：总线和 I/O 的角色先分清

本册的十一个关键词

- **总线**：连接多个主设备和目标设备、承载地址、数据与控制信号的共享或点对点互连。
- **主设备/目标设备**：当前发起事务的一方和被寻址、响应事务的一方；同一设备在不同事务中可能换角色。
- **事务**：从请求或授权开始，到数据传送和结束确认的一次完整总线操作。
- **仲裁**：多个主设备同时请求总线时，决定谁暂时获得驱动权的过程。
- **握手**：用 request/acknowledge、ready/wait 等信号确认双方何时发送和采样。
- **I/O 控制器**：夹在 CPU/总线和外设之间的硬件，提供数据、状态、命令寄存器并吸收速度差。
- **轮询 polling**：CPU 反复读取状态寄存器，直到设备报告就绪。
- **中断 interrupt**：设备提出请求，CPU 在允许的指令边界保存断点并转入中断服务程序。
- **DMA**：直接存储器访问；DMA 控制器代表设备取得总线，把一块数据直接搬到主存或从主存搬出。
- **DREQ/IRQ**：DREQ 是 DMA 传送请求，IRQ 是要求 CPU 处理事件的中断请求；两者都

不是“已经完成”的证明。

- **MMIO**：把设备寄存器放进内存地址空间；访问是否可缓存、可重排和可用的宽度由平台属性规定。

目录

1 术语先行：总线和 I/O 的角色先分清	1
2 总线是协议，不是一捆静态导线	4
3 同步、异步与分离事务	4
4 仲裁：选择共享资源的临时控制者	4
5 带宽题先写完整事务	4
6 I/O 接口把设备变成寄存器状态机	5
7 三种控制方式：谁等待、谁搬数据、谁通知	5
8 中断硬件路径与软件交接	5
8.1 可屏蔽中断与硬件响应条件	6
8.2 硬件自动入口 vs ISR 软件服务	6
8.3 响应优先级 vs 处理优先级	7
9 DMA：把搬运循环下沉到控制器	7
10 贯穿母例：DMA 读取一个块	7
11 五列概念边界	8
12 做题控制协议	8
13 本册与 CPU/设备的接口	8
14 知识地图：从请求到完成证据	9
15 补充细节：总线协议的可计算细节	9
16 四种定时协议与异步互锁	9
17 仲裁实现的线数—延迟—公平性三角	10

18 I/O 接口与端口状态机的完整边界	10
19 中断的五阶段与 CPU 成本	10
20 DMA 控制器与三种传送方式	11
21 通道是 DMA 之上的组织层	11
21.1 I/O 方式的统一定量骨架	11
21.2 总线标准与并行/串行演进	12

2 总线是协议，不是一捆静态导线

共享互连至少包含 initiator/master、target/slave、arbiter 和 interconnect。一次事务通常经过请求、仲裁、地址/命令、一个或多个 data beat、response 与释放。DMA 控制器获授权后也可成为 initiator；“CPU 永远是主设备”不是不变量。

地址、数据与控制信息承担不同责任：地址标识目标，数据线宽度限制每 beat 原始数据量，控制/响应表达 read/write、byte enable、ready、error、request/grant 等协议状态。地址线分时复用不会自动增加数据带宽。

3 同步、异步与分离事务

同步总线按共同 clock 采样，慢目标可插入 wait state；异步总线靠 request/acknowledge 握手建立因果关系。二者描述**定时协议**，串行/并行描述**物理传输**，不能互推快慢。

分离事务把 request 与 response 解耦，长延迟时可释放通道服务其他请求，但需要事务 ID、缓冲和匹配。是否允许多个 outstanding request 是协议属性，不由“异步”一词推出。

4 仲裁：选择共享资源的临时控制者

仲裁只决定哪个请求者在当前时段获得发起权，不决定 OS 层的设备归属。固定优先级响应关键设备快，却可能饥饿；轮转改善公平性，却增加状态与最坏等待；链式、独立请求或分布式实现还要比较线路成本、仲裁延迟和故障范围。

机制	关键状态	优势	风险
固定优先级	pending 与 priority	关键请求低延迟	低优先级饥饿
轮转	last grant / pointer	长期公平	状态与最坏等待增加
链式查询	grant 传播位置	线路少	延迟累积、物理顺序固化
独立请求	每主设备 request/grant	并行判优快	线路和逻辑成本高

5 带宽题先写完整事务

理论峰值为

$$BW_{peak} = \frac{bytes}{beat} \times \frac{beats}{cycle} \times frequency.$$

有效带宽还要扣除仲裁、地址/响应阶段、wait、方向切换、编码与空闲。若一个事务搬运 D 字节、总耗时 T ，稳定口径是 $BW_{effective} = D/T$ 。频率、总线周期、数据周期与设备时间必须分开列。

最小反例

64 位、1 GHz 只给出每 beat 上限与时钟条件；若每次数据前有地址阶段、目标插入等待，或并非每周期都有 beat，有效带宽就不是自动的 8 GB/s。

6 I/O 接口把设备变成寄存器状态机

控制器通常暴露 data/FIFO、status、control/command，以及可选地址、计数、描述符和 interrupt mask。软件写 command 只是启动；busy、ready、done、error 才描述后续状态。

memory-mapped I/O 用普通地址空间和 load/store；isolated I/O 用独立端口空间和专用指令。差别在软件可见寻址契约，不等于设备性能。MMIO 是否可缓存、能否重排和访问宽度须服从 ISA/平台规则。

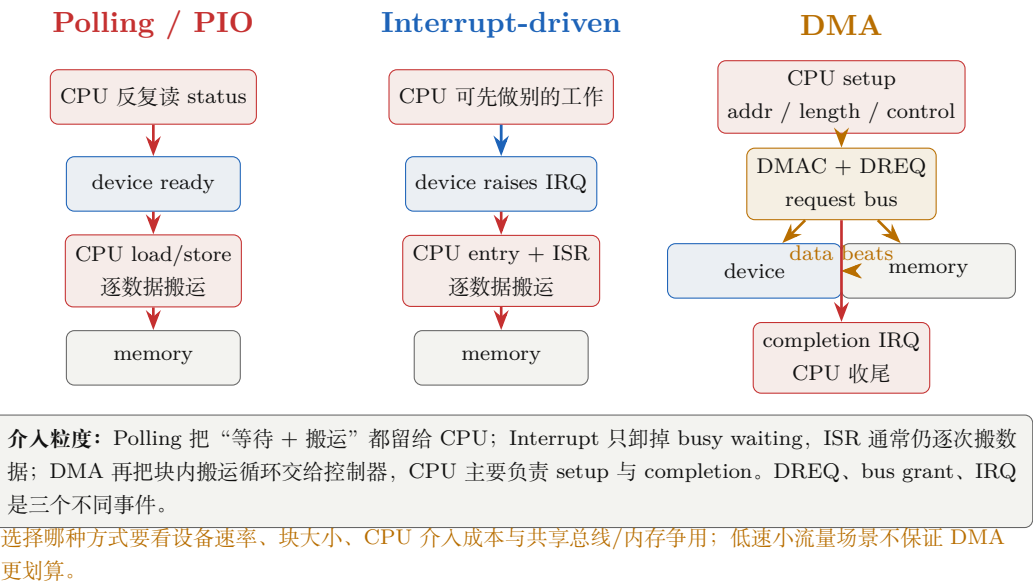
7 三种控制方式：谁等待、谁搬数据、谁通知

方式	CPU 等待	数据路径	完成发现
Polling / PIO	循环查询状态	device ↔ CPU register ↔ memory	主动读取 done
Interrupt-driven	可执行其他任务	通常仍由 CPU 指令搬运	设备异步请求入口
DMA	只配置与收尾	device ↔ memory	块完成/错误通知

中断消除的是持续 busy waiting，不自动消除 ISR 和搬运成本；DMA 下沉的是传输循环，不代表 CPU、Cache、内存和互连没有成本。

Polling、Interrupt、DMA：真正变化的是 CPU 等待、搬运与通知责任

三种方式都需要设备接口状态；升级不是“设备本身更快”，而是逐步减少 CPU 在等待与逐数据搬运上的介入频率。



8 中断硬件路径与软件交接

device event → pending → mask/priority → precise boundary → architectural entry state → vector.

外部中断与当前指令异步；异常通常与当前指令同步相关。CPU 是否自动关中断、保存哪些最小状态、向量怎样形成均由 ISA 规定。完整通用寄存器保存、驱动服务、确认设备、唤醒进程和调度属于软件。

中断控制器的 pending、enable/mask、priority 和 in-service 状态必须分开：请求到达不等于立即接受，接受不等于 handler 已完成，handler 返回也不必自动清除设备源头。

8.1 可屏蔽中断与硬件响应条件

对可屏蔽外部中断，硬件在响应前必须同时满足三个硬性判据：

- 1. **请求提出**：中断源接口电路确实已将中断请求信号置位；
- 2. **允许响应**：CPU 当前处于开中断状态（PSW 中 IF = 1）；
- 3. **指令边界**：当前执行指令已到达允许响应的周期边界（通常为执行周期的最后时刻）。

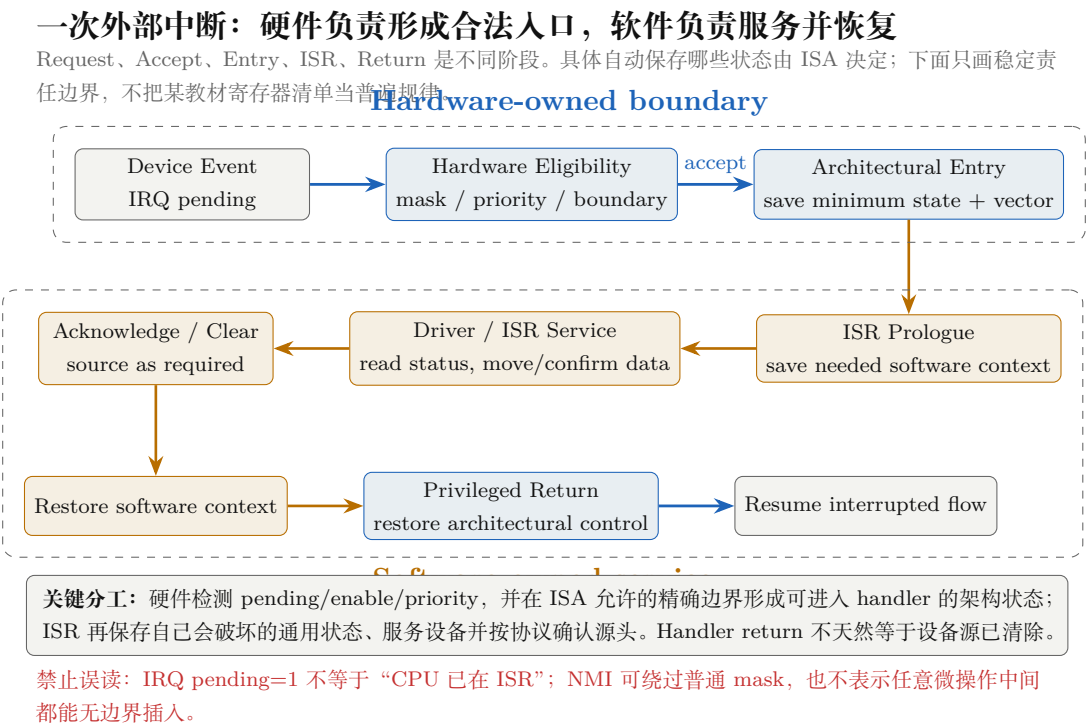
不可屏蔽中断（NMI）可绕过 IF 屏蔽位，但仍须在指令执行边界响应；同步异常则由当前指令执行直接触发。

8.2 硬件自动入口 vs ISR 软件服务

硬件与软件的交接边界必须绝对分清：

- **硬件中断隐指令 (Hardware Auto Actions)**：关中断 → 保存断点（PC 与 PSW）→ 形成中断服务程序入口地址并送入 PC；
- **软件 ISR 服务 (Software Driver Service)**：保存通用寄存器现场 → 开中断（若允许嵌套）→ 执行具体设备 I/O 服务 → 关中断 → 恢复通用寄存器 → 执行特权返回指令 `iret`。

向量中断中，向量号索引向量表得到入口地址；非向量中断由软件轮询源头。注意：“向量地址”（表项地址）与“中断向量”（表项内容/入口地址）不可混为一谈。



8.3 响应优先级 vs 处理优先级

- **响应优先级**：由硬件排队器或仲裁电路决定，不可软件动态更改；
- **处理优先级**：由中断屏蔽字（Mask）在 ISR 中动态设定，可改变多重中断时的服务抢占次序。

若允许嵌套，屏蔽字至少应屏蔽“处理优先级不高于当前级”的中断（含自身），位序必须按题面编号严格重排。

硬件边界

本 Topic 的最后一个稳定动作是“按 ISA 规定形成可进入软件的体系结构状态”。“唤醒哪个任务、是否重试请求、怎样处理错误”不写成硬件动作。

9 DMA：把搬运循环下沉到控制器

CPU 先配置 source、destination、length、direction/control 或 descriptor；DMAC 获得设备和总线条件后发起 transaction，逐 beat 更新地址与计数，完成或错误时置状态并通知 CPU：

Setup → DREQ → Arbitrate → Transfer → Count/Address Update → Complete.

burst 一次授权连续传输，吞吐高但占用长；cycle stealing 每次占少量周期，在 CPU 干扰与吞吐间折中；transparent/interleaved 利用约定时隙，依赖具体时序。DREQ 是设备到 DMAC 的传输请求，不是 CPU 中断；总线 grant 也不是上下文切换。

scatter-gather 可由描述符链减少大块非连续传输的重复配置，但描述符建立、地址映射、缓存一致性和完成处理仍产生 CPU/系统成本，因此“DMA 对任意传输恒为 $O(1)$ CPU 开销”不是稳定结论。

10 贯穿母例：DMA 读取一个块

1. 软件准备设备可用的目标地址、长度和方向，并按平台规则处理映射/一致性；
2. CPU 写设备/DMAC 控制寄存器，控制器进入 busy；
3. 设备产生 DREQ，DMAC 作为潜在 initiator 请求互连；
4. arbiter 授权后，DMAC 与内存/设备完成若干 beat，更新地址和剩余计数；
5. 共享内存或总线冲突表现为等待，CPU 是否同时运行取决于其当前资源需求；
6. 计数归零或错误使控制器写 completion state，并产生可屏蔽的 interrupt request；
7. CPU 在精确边界接受中断，形成 ISA 规定的入口状态；
8. 软件确认状态、解除映射、完成请求或唤醒任务。

11 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
request	≠ transaction	提出需求 vs 已获权并执行协议	漏仲裁与等待
bus grant	≠ interrupt accept	互连所有权 vs CPU 控制流入口	把暂停访存写成 ISR
DREQ	≠ IRQ	设备请求 DMAC 搬运 vs 控制器通知 CPU	错画信号方向
interrupt entry	≠ OS service	ISA 最小入口 vs 软件处理与唤醒	硬件越界拥有 OS
peak bandwidth	≠ effective bandwidth	理想 beat 速率 vs 完整事务有效数据率	漏协议开销
MMIO address	≠ memory data	设备寄存器契约 vs 普通可缓存对象	错用缓存/重排

12 做题控制协议

1. 标 initiator、target、数据方向、共享资源与 completion 条件；
2. 把 request、arbitration、address/control、data beat、response 分开；
3. 逐阶段写资源控制者和允许重叠的 CPU/设备工作；
4. 带宽先算完整事务有效数据/总时间，再与峰值比较；
5. I/O 方式题回答谁等待、谁搬数据、谁通知；
6. 中断题分 pending、accept、entry、software service、return；
7. DMA 题分 setup、transfer、completion，并单列一致性和 OS handoff。

压缩成第一动作

看到总线或 I/O 题，先问：当前谁拥有控制权？当前事件只是 request，还是已经形成 transaction 或 completion evidence？

13 本册与 CPU/设备的接口

从请求到完成证据

本册负责共享线路、控制器状态、仲裁、事务和完成证据。中断解决“完成后怎样通知 CPU”，DMA 解决“数据怎样批量搬运”；两者可以组合但不能互相替代。请求队列、完成后的阻塞/唤醒和设备调度属于软件。

本册 Cost 要把事务有效数据、地址/控制/等待周期、仲裁、方向切换和可重叠阶段写全；总线峰值、有效带宽、设备 latency 与 CPU 指令时间不是同一个量。若访问是 MMIO，属性可能绕过普通 Cache，但仍需回到 CO-07/B02 的权限与地址接口。

14 知识地图：从请求到完成证据

考点	本册机制	接口
总线结构/仲裁/定时	角色、事务、握手、有效带宽	CO-03/05
I/O 接口与编址	寄存器状态机、MMIO/isolated I/O	CO-02
轮询/中断	等待、搬运、通知；硬件精确入口	X-B01/X-B03
DMA	setup、仲裁、transfer、completion	OS-05/X-B03

一句话

总线题追踪“谁在何时拥有共享线路”，I/O 题追踪“谁推进状态、谁搬数据、什么证据证明完成”；中断与 DMA 都只是跨层 handoff，不替 OS 完成软件责任。

本册把总线结构、I/O 控制方式、中断和 DMA 统一到“请求、授权、传输、完成”的状态机中。具体总线标准的电气参数、设备驱动策略和操作系统调度必须以题设或平台说明为准。

15 补充细节：总线协议的可计算细节

一次总线事务可按考纲的四阶段复述为：**申请/分配**（主设备提出请求并由仲裁授权）、**寻址**（发出从设备地址与命令）、**传输**（一个或多个数据 beat）、**结束**（撤销信号并释放总线）。在单主设备系统中第一、四阶段可以被硬件常态化隐藏，但 DMA、多处理器或多个 I/O 主设备出现后不能省略。突发传送只发首地址并自动递增后续地址，非突发传送则每个数据都重新付出寻址开销。

信号组/对象	语义	易错边界
数据线	被搬运的数据、状态字或中断类型号等内容	“命令字”是内容，仍走数据线；请求/应答是控制信号
地址线	目标存储单元或 I/O 端口编号	地址/数据复用减少引脚，不会自动提高数据阶段带宽
控制线	读写、时钟、BR/BG、ready/error、request/ack 等协议状态	控制线有出有人，不应按“全部单向”处理

总线还可以按距离分成片内、系统和通信/外部总线；不同速度等级之间由桥接器做协议与速率转换。总线宽度 W （字节）和工作传输频率 F 决定理论峰值 $BW = W \times F$ 。若题目给的是基础时钟 f ，且每周期有 k 个数据传送，则 $F = kf$ ；若给的是 MT/s 或 GT/s 这类有效传输率，就不要再次乘 k 。带宽只描述数据阶段的上界，地址、仲裁、等待和方向切换应放进另一个“完整事务有效速率”计算。

16 四种定时协议与异步互锁

同步通信让所有动作对齐公共时钟，短距离且部件速度相近时控制简单；慢设备可插入固定的 wait state，但固定时钟仍按约定的最慢时间运行。半同步保留时钟边沿，在从设备来不及由 WAIT 拉长一个或多个整周期，适合速度有差异而仍希望保留同步实现的系统。

异步通信没有公共时钟，依靠 request/acknowledge 握手。按信号撤销是否等待对方，可用互锁处数判断三类：不互锁（双方都按约定延时撤销，最快但最不可靠）、半互锁（主设备等回答后撤销请求，从设备不等请求撤销）和全互锁（请求等回答，回答再等请求撤销，最可靠但握手最长）。若单程传播及判别时间为 t 、从设备准备时间为 T_s ，在题面把 t 明确为单程时，典型关键路径可写成

$$T_{\text{no-lock}} \approx t + T_s, \quad T_{\text{half-lock}} \approx 2t + T_s, \quad T_{\text{full-lock}} \approx 4t + T_s.$$

不要把一次往返时间误当单程，也不要按 T_s 按互锁次数重复计算。这里的“速度—可靠性”排序只属于握手协议，不是串行/并行传输的排序。

17 仲裁实现的线数—延迟—公平性三角

集中式仲裁的三类经典实现可以按授权信息的编码理解：链式查询把授权逐跳传递，控制线数近似常数但响应慢、优先级由物理顺序固定且容易因断链影响下游；计数器定时查询广播设备编号，控制线约为 $\lceil \log_2 n \rceil + 2$ ，可从 0 开始实现固定优先级，也可从上次终点继续实现轮转；独立请求为每个主设备配置 BR/BG，线数约 $2n + 1$ ，判优最快且最灵活。分布式仲裁则把仲裁逻辑分散在各主设备，通过共享仲裁号比较避免单点仲裁器，代价是每个设备都需额外逻辑。

读图判仲裁

先找“谁产生 BR、谁产生 BG、谁决定赢家”。若授权沿设备链传递，是链式；若设备地址在公共线上逐个比较，是计数器查询；若每台设备有独立 BR/BG，是独立请求；若不存在中央决策点而各设备同时比较仲裁号，是分布式。仲裁只决定临时总线控制者，不决定设备驱动归属。

18 I/O 接口与端口状态机的完整边界

外设与 CPU 存在速度、信号格式和控制协议三重不匹配，CPU 实际对话的对象是设备控制器而非设备本体。接口向 CPU 暴露的端口通常分为数据/FIFO、状态和命令/控制三类；DMA 还需要把主存地址、长度、方向或描述符交给控制器。无条件传送的设备可以在约定时刻直接传输；条件传送则必须先读 ready/busy 等状态，再读写数据端口。

程序查询方式中，CPU 反复读取状态位直到就绪，设备准备数据期间两者串行；定时查询的 CPU 占比可按

$$\rho_{\text{poll}} = \frac{R/u \times C_{\text{poll}}}{f}$$

估算，其中 R 是字节率， u 是每次端口传送的字节数， C_{poll} 是一次查询消耗的周期数， f 是主频。 u 来自端口/缓冲寄存器宽度，不能默认等于 1 字节。I/O 指令能直接改变设备状态，通常属于特权操作；MMIO 是否允许缓存、重排或宽度变化则必须服从 ISA/平台属性。

19 中断的五阶段与 CPU 成本

程序中断可拆成“请求、判优、响应、服务、返回”五阶段。接口在设备就绪时置请求触发器；中断控制器按 pending、mask/enable、priority 和 in-service 状态判优；CPU 在符合 ISA 的精确边界执行中断隐指令，保存断点、形成入口并交给 ISR；ISR 再保护现场、读取/写入端口、确认设备、恢复现场并执行返回指令。请求到达不等于已经接受，handler 返回也不一定自动清除设备源头。

若每次服务涉及 m 个周期、设备每秒产生 R/u 次传送，单次中断 CPU 占比为

$$\rho_{irq} = \frac{(R/u) \times m}{f}.$$

中断方式的价值是释放设备准备期间的等待时间，而不是保证单次搬运比查询便宜；当设备到达间隔小于一次中断响应加服务时间时，中断在物理上就无法及时消费数据，应升级到 DMA 或更强的缓冲结构。

20 DMA 控制器与三种传送方式

DMA 控制器的最小寄存器集合可从“独立完成一块访存”推出：AR 保存当前主存地址并按 DR 字节数递增，WC 保存剩余传送个数并递减，DR 作为设备侧与总线侧的过渡缓冲，CR 保存方向、启动和状态；还需要总线请求/授权逻辑和完成中断电路。完整流程仍是预处理（CPU 设置参数）、数据传送（DMAC 发地址与读写信号并更新 AR/WC）、后处理（完成/错误中断）。

CPU 与 DMA 争用主存时有三种经典方式：**停止 CPU 访存**控制简单，适合高速设备的成组传送但 CPU 整段停顿；**周期挪用**每次只占一个或几个存储周期，设备两个数据之间的间隔必须足以摊平申请/建立/归还总线的开销；**交替访存**把一个 CPU 工作周期拆成 CPU 与 DMA 两个分周期，CPU 几乎不等待但要求 CPU 工作周期长于主存存取周期，硬件最复杂。三者不是抽象的优劣排序，必须根据题目给出的设备周期、主存周期和块长度判断成立条件。

21 通道是 DMA 之上的组织层

通道不是普通 DMA 控制器的别名。DMA 的行为由 AR/WC/CR 等参数固定，通道则拥有自己的通道指令系统，能从主存取出通道程序，自主完成设备选择、块组织和错误处理。层级可写为

$$\text{CPU} \rightarrow \text{Channel} \rightarrow \text{Device Controller} \rightarrow \text{Device}.$$

字节多路通道按字节交叉服务大量低速设备；选择通道按块独占通道服务少数高速设备；数组多路通道按块交替并允许多个子通道，折中吞吐与并行。通道属于 I/O 系统的扩展知识，若题目只考 DMA，仍应把它作为“DMA 已卸掉搬运、通道进一步卸掉组织”的边界，而不把 OS 请求队列写成硬件动作。

21.1 I/O 方式的统一定量骨架

若设备产生速率为 R B/s、每次端口/缓冲传送 u B，则程序查询和程序中断的介入频率都约为 R/u ；查询的 CPU 占比可写成 $(R/u)C_{poll}/f$ ，中断的占比为 $(R/u)m_{irq}/f$ ，其中 m_{irq} 包含响应、保护现场、搬运、恢复和返回。DMA 以块大小 B_s 介入，设置和完成的次数约为 R/B_s ，占比应写成

$$\rho_{DMA} = \frac{(R/B_s)m_{setup+finish}}{f},$$

并把 Cache 一致性、描述符、仲裁和后处理成本单列。一次块传送通常是 CPU 预处理一次、完成中断/后处理一次，所以“DMA 介入次数与数据个数无关”只在块大小和完成模型已声明时成立。

查询、中断、DMA 和通道可沿“CPU 介入程度”排列：查询全程等待，中断通常每个数据/小单元介入，DMA 每块配置与收尾，通道每个 I/O 任务只需启动和完成交接。升级不是绝对的“越快越好”：低速设备可能以查询的硬件成本最优；高速设备若 $T_{device\ gap} < T_{irq\ response} + T_{ISR}$ ，中断在物理上会丢数据，应采用 DMA 或更强缓冲。

陌生总线/I/O 题的停止条件

完成一题前必须写出：当前 initiator/target、请求对象（CPU 执行权还是总线控制权）、事务四阶段、数据路径、完成证据、异常/中断入口和软件接手位置。若只写“DMA 更快”“中断更高效”而没有给出块大小、端口宽度、仲裁与等待周期，定量结论还未成立。

21.2 总线标准与并行/串行演进

总线标准不是只规定带宽，而是同时规定四类接口契约：机械特性（插头、插槽、引脚几何）、电气特性（方向、电平和信号完整性）、功能特性（每根线承担地址/数据/控制哪种语义）和时间特性（何时有效、何时采样）。缺少任何一层，模块都可能“插得上却不能通信”。

低频短距离下，并行线一次传多位；高频长距离下，走线长度差、串扰和时钟偏移会使最快/最慢位无法在同一采样窗内稳定到达，频率越高问题越严重。串行差分链路以较少的位间对齐约束支持更高频率，并可从数据跳变恢复时钟；多条独立 lane 再把吞吐量线性叠加。因此“并行一定比串行快”不是稳定结论，必须连同距离、频率和信号完整性条件判断。

从共享并行总线转向点对点串行链路，还减少了多设备负载、共享仲裁和带宽分摊。PCIe 的 $\times n$ 表示 n 条 lane，而不是一条 n 位并行数据总线；USB、SATA 等也体现串行链路的工程取舍。具体版本、编码比例和标准速率属于题设实例，不应升级为 408 的固定常数，但“并行 $\rightarrow$ 串行、共享 $\rightarrow$ 点对点”的因果链应保留。